Callback-Grenze, Workqueue und Beobachtbarkeit

Ausführungskontexte trennen, Überlast sichtbar machen und event status gezielt nutzen

3.11

Zeitlinie der Ereigniskette

AUSFÜHRUNGSKONTEXTE

```
Shell → Stimulus → Callback → Event Service
```

Shell-Handler	prüft Argumente und startet den Stimulus
Stimulus-Worker	ändert den emulierten GPIO-Pegel im Threadkontext
GPIO-Callback	erfasst Zustand und signalisiert nicht blockierend
Event-Worker	führt Fachlogik aus und aktualisiert Statistiken
event status	liest einen konsistenten Diagnoseschnappschuss

Jede Aktion besitzt einen klaren Kontext. Potenziell blockierende Arbeit bleibt im Threadkontext.

Die Callback-Grenze

01	**Erfassen**	Pegel, Zeitmarke und Rohzähler mit konstantem Aufwand sichern
02	**Signalisieren**	Work-Item, Queue oder atomare Markierung ohne Warten auslösen
03	**Vermeiden**	Shell-Ausgabe, Wartezeiten, Speicherallokation und Fachlogik

Die Callback-Laufzeit bleibt unabhängig von der Komplexität der späteren Verarbeitung.

Synchronisation nach Datenart

ATOMAR

Einzelne Zähler und Flags

atomic_inc() für Rohzähler

atomic_set() für Zustandswerte

in Thread und ISR nutzbar

KURZER DATENSATZ

Zusammengehörige Felder

kurze kritische Sektion

Kontextgrenzen beachten

keinen Mutex aus ISR verwenden

EREIGNISQUEUE

Reihenfolge erhalten

k_msgq für Datensätze

Kapazität ist begrenzt

Überlauf explizit zählen

Die Synchronisation folgt Datenform und Ausführungskontext – nicht persönlicher Vorliebe.

Callback: erfassen und weiterreichen

GPIO-CALLBACK

```
atomic_inc(&raw_count);
atomic_set(&last_pins, pins);
atomic_set(&last_cycle,
           k_cycle_get_32());

(void)k_work_submit(&event_work);
```

01 Nicht blockierend

Der Callback erfasst nur Diagnosewerte und stößt die Verarbeitung an.

02 Bewusste Semantik

Ein bereits anstehendes Work-Item kann mehrere Rohereignisse zusammenfassen.

k_work_submit() verlagert Verarbeitung, garantiert aber nicht automatisch ein Work-Item pro Ereignis.

Verlust, Zusammenfassung und Überlast

01	`Verlustfrei zählen`	ein Datensatz pro Ereignis; Queue-Kapazität und Überlaufpfad festlegen
02	`Zusammenfassen`	anhängiges Work verarbeitet den neuesten Zustand; Roh- und Verarbeitungszähler trennen
03	`Gezielt verwerfen`	bei Sättigung klarer Fehlerpfad; lost_count oder overload_count erhöhen

Koaleszierung und Verlust sind Teil des Vertrages und müssen in event stats sichtbar sein.

event status als Diagnosefläche

Sequenz	Typ, aktiv/inaktiv und verbleibende Schritte
Letzter Fehler	Rückgabewert und Herkunft des letzten Fehlers
Letztes Ereignis	Zeitmarke, Pegel und optional verstrichene Zeit
Zähler	raw, processed, coalesced und lost getrennt
Worker-Zustand	pending, busy und Queue-Füllstand soweit verfügbar

Diagnosemarken und Statistiken ersetzen umfangreiche Ausgabe im Callback.

Interaktiv prüfen, automatisiert absichern

INTERAKTIV

Shell für Exploration

Stimuli gezielt auslösen

event status beobachten

Grenzfälle variieren

Hypothesen schnell prüfen

AUTOMATISIERT

Semantik beweisen

definierte Sequenzen

Zählerstände prüfen

Fehlerpfade wiederholen

Timingtoleranzen festlegen

KONTEXTREVIEW

Blockierung bewerten

Shell-Handler: Thread

Stimulus: Workqueue

Callback: kurz

Fachlogik: Thread